云屋三面前端（+ AI Agent 开发）· 项目考核

小满商城

文件说明书

每个文件做什么，它们是怎么连起来的

技术栈：原生 HTML5 / CSS3 / ES6 模块 + Node.js 零依赖后端

商品数据：26 件生活好物，6 个分类；数据放在服务端 JSON 文件里

覆盖流程：浏览 → 搜索筛选 → 详情 → 加购 → 结算 → 模拟支付 → 订单结果

怎么读这份说明书

先看第 1、2 节把项目跑起来；再看第 3 节理解一次请求的旅程；第 4–8 节按文件查作用；第 9、10 节是讲给别人听时要用的链路和知识点。

2026 年 10 月 · 项目源码与本文档同步维护

阅读路线

这份说明书不是 API 手册，而是“带你看懂自己的项目”。不同目的可以走不同路线。

你的目的	看哪几节	读完能得到什么
想赶紧跑起来看看	第 1、2 节	知道装什么、敲什么命令、打开哪个地址
想讲清前后端怎么配合	第 3、9 节	能画出一条请求从点击到渲染的完整路径
想弄懂每个文件	第 4–8 节	知道每个文件负责什么、为什么这样拆
想准备面试/答辩	第 10、11 节	知识点对照、演示顺序、常见追问

一个先记住的结论

这个项目里，浏览器只做三件事：**收集用户操作、发请求、把返回的数据画出来。**价格、库存、订单号、金额这些“算错了要出事”的东西，全部由服务端决定。前后端之间只通过 14 个 HTTP 接口说话，接口的格式在第 3 节和第 9 节会反复出现。

项目里的文件命名规则：前半段英文说明“这是什么”，后半段中文说明“它干什么”，例如 **cart-购物车服务.js**、**checkout-结算下单.js**。这样在编辑器里搜索、在答辩时指给别人看，都不用先解释缩写。

六个会反复出现的词

词	在这个项目里的意思
信封	后端返回的固定三字段：code 表示成功或错误，message 是给人看的中文，data 才是数据
路由	一张“URL + 方法 → 函数”的映射表；/api/products 进来，就交给商品接口那个函数
服务层	只写业务规则（筛选、算钱、扣库存），不碰 req / res，所以接口和脚本都能调用它
统一错误	业务代码只管 throw HttpError，router 负责把它翻译成状态码和 JSON，不用每处写 res
发布订阅	购物车状态变了，就通知所有订阅者（角标、抽屉、弹窗）一起重绘
骨架屏	数据还没回来时先画几个灰色占位块，避免页面空白或内容突然跳动

一、项目速览

小满商城是一个可以完整走通的购物网站：首页浏览 → 搜索/筛选 → 商品详情 → 加入购物车 → 修改数量 → 填写收货信息 → 提交订单并模拟支付 → 查看订单结果。它不是几张静态页面的拼图，列表、详情、购物车、订单的数据都来自后端接口。

项目指标	数量	说明
页面	4 个	首页、商品详情、我的订单、404
后端接口	14 个	商品 5 个、购物车 5 个、订单 4 个
商品数据	26 件 / 6 类	价格 19–1699 元，库存、销量、标签各不相同
第三方依赖	0 个	后端只用 Node 自带的 http / fs / path / crypto
数据文件	4 个	商品与分类随项目提供；购物车与订单在运行时生成
前端构建步骤	无	浏览器直接加载 ES Module，不需要打包

为什么用原生三件套

考核允许原生或框架。这个项目只有 4 个页面、26 件商品，用框架反而要先解释构建工具、路由和状态库；用原生可以把注意力放在“请求怎么发、状态怎么变、页面怎么重绘”这三件最基础的事上。没有构建步骤也意味着：打开目录就能读，启动服务就能跑，答辩时不会卡在环境上。

项目刻意保留的“手写感”

· 路由是自己写的：一个 add(method, pattern, handler) 加一段路径匹配，能看清 URL 是怎么变成函数调用的。

· 状态管理是自己写的：一个几十行的发布订阅，购物车一变，角标、抽屉、结算弹窗同时更新。

· 数据存储是自己写的：JSON 文件当表，读进内存、改完落盘，够用且没有黑盒。

二、运行与目录地图

环境要求只有一条：Node.js 18 或更高版本。项目没有 npm 依赖，所以不需要 npm install。

```
cd xiaoman-mall
npm start
# 或者：node server/index-服务入口.js

# 终端出现下面这行就成功了
小满商城已启动：http://127.0.0.1:3000
```

浏览器打开 http://127.0.0.1:3000 即可。换端口用环境变量 PORT，例如 PowerShell：$env:PORT=4000; npm start。

目录地图（缩进表示层级）

下面每一行都是一个真实存在的文件或目录，右列是它存在的理由。

文件 / 目录	作用
server/	**后端代码，只在 Node 里运行，浏览器访问不到**
index-服务入口.js	程序入口：创建 HTTP 服务，分发 /api 与静态文件
router-迷你路由.js	极简路由：注册路径、匹配 :id、解析 query 与 body
data/	**数据目录，相当于“数据库”**
products-商品数据.json	26 件商品的完整信息
categories-商品分类.json	6 个分类及说明
carts-购物车数据.json	运行时生成，存每辆购物车
orders-订单数据.json	运行时生成，存每一笔订单
lib/	**与业务无关的通用工具**
response-统一响应格式.js	成功/失败信封、HttpError、sendJson
store-JSON数据存储.js	读写 JSON、内存缓存、表名到文件名的映射
static-静态文件服务.js	返回 HTML/CSS/JS/SVG，处理 404 与 304
services/	**业务规则层：搜索、购物车、订单都在这**
catalog-商品查询服务.js	关键词、分类、价格、排序、分页、推荐
cart-购物车服务.js	加购、改数量、删除、清空、算小计与总额
orders-订单服务.js	下单校验、算运费、生成单号、扣库存、支付/取消
routes/	**HTTP 与 service 之间的薄薄一层，只取参数**
products-商品接口.js	5 个商品相关 GET 接口
cart-购物车接口.js	5 个购物车接口（GET/POST/PATCH/DELETE）
orders-订单接口.js	4 个订单接口（创建/查询/支付/取消）
public/	**前端代码，浏览器直接加载**
index-首页.html	首页：导航、Hero、分类、推荐、列表、抽屉、结算弹窗

文件 / 目录	作用
product-商品详情页.html	详情页：图片、价格、规格、介绍、相关推荐
orders-我的订单页.html	订单结果页 + 历史订单
404-页面不存在.html	服务端找不到页面时返回的静态错误页
css/	**5 个样式表，按层加载**
js/	**9 个 ES 模块，一个文件一件事**
assets/products/	**26 张商品封面 SVG**
scripts/gen-covers-生成商品封面.js	按分类配色批量生成 SVG，并把路径写回商品 JSON
docs/	**接口文档、演示脚本、截图、本说明书**
package.json	项目名、启动脚本、Node 版本要求
README.md	项目介绍、快速开始、考核点对照
.gitignore	忽略依赖、日志、运行时数据

为什么 data 里两个文件是“生成”的：购物车和订单属于运行时数据，第一次写入时程序会自动创建；删掉它们不会影响启动，反而能得到一个干净的演示环境。

按功能找文件：四个入口

· 想改搜索、筛选、排序 → 先看 services/catalog-商品查询服务.js，再看 js/home-首页逻辑.js 里怎么把条件拼成请求。

· 想改加购、数量、删除 → 先看 services/cart-购物车服务.js，再看 js/cartdrawer-购物车抽屉.js 怎么渲染。

· 想改金额、运费、下单规则 → 先看 services/orders-订单服务.js，再看 js/checkout-结算下单.js 怎么提交。

· 想改颜色、间距、卡片长相 → 先改 css/base 里的变量，再改 components；页面专属的才动 pages。

实在找不到时，打开浏览器 Network 面板，点一次页面上的操作，看请求路径（例如 /api/cart/items），再到 routes 目录里搜这个路径，就能顺着找到对应的 service。

三、一次请求是怎么走的

这是整个项目最重要的一张图。看懂它，就看懂了前后端如何配合。以“首页加载商品列表”为例：

```
[浏览器] 打开首页
  |
  | ① home-首页逻辑.js 读取筛选条件，调用 api.products.list()
  v
[api-请求封装.js] 拼查询字符串，fetch 发请求，带 12 秒超时
  |
  | ② GET /api/products?page=1&pageSize=8&sort=...
  v
[index-服务入口.js] 看到 /api 前缀，交给 router；否则读 public 下的文件
  |
  | ③ router.handle() 匹配路径，解析 query / body / params
  v
[routes/products-商品接口.js] 只做一件事：把 query 交给 service
  |
  | ④ catalog.query(query)
  v
[catalog-商品查询服务.js] 关键词 -> 分类 -> 价格 -> 排序 -> 分页
  |
  | ⑤ 读数据
  v
[store-JSON数据存储.js] 从内存缓存或 JSON 文件取商品
  |
  | ⑥ 逐层返回，最后包成 { code: 0, message: "ok", data: {...} }
  v
[api-请求封装.js] 检查 code，等于 0 就把 data 交给页面
  |
  | ⑦ home-首页逻辑.js 生成卡片 HTML，替换骨架屏
  v
[页面更新] 商品卡片、结果数量、分页器同时出现
```

这条链路里有三个约定

- **统一信封**：不管成功失败，后端都返回 code / message / data 三个字段。前端只判断 code，错误提示直接用 message，不需要为每个接口写一套解析。
- **统一错误**：service 里发现问题就 throw HttpError(状态码, 业务码, 中文原因)，router 捕获后转成 JSON。业务代码不用关心 res 怎么发。
- **分层不越界**：routes 不写业务，services 不碰 req/res，lib 不认识商品和订单。所以换一种前端、加一个 AI 工具，都可以复用 services。

四、后端文件说明

后端按“入口 → 路由 → 接口 → 服务 → 存储”五层拆开，每一层只依赖下一层。读代码时可以从 index 一路往下，也可以从任意接口反着找。

文件	作用与关键点
server/index-服务入口.js	创建 http.createServer；/api 走路由，其余走静态文件；启动后打印地址；端口被占用时给中文提示，而不是一串堆栈。
server/router-迷你路由.js	add(method, pattern, handler) 注册；把 /api/products/:id 转成正则并提取 params；解析 URL、query、JSON body（上限 1MB）；捕获 HttpError 转 JSON，未知异常转 500；顺带处理 CORS 与 OPTIONS。
server/lib/response-统一响应格式.js	ok() / fail() 统一信封与状态码；HttpError 把“状态码 + 业务码 + 中文原因”打包成一个异常；sendJson 统一 Content-Type。
server/lib/store-JSON数据存储.js	FILES 表把 products/carts/orders 映射到中文文件名；read 先查内存缓存，没有再读文件；write 先改内存再同步落盘；update 封装“读—改—写”。
server/lib/static-静态文件服务.js	按 URL 找 public 下的文件；MIME 表决定 Content-Type；/ 默认到首页；找不到返回 404 页面；用 path.resolve 防止 ../ 目录穿越；支持 If-Modified-Since 304。
server/services/catalog-商品查询服务.js	toNumber 校验价格参数；SORT_RULES 管五种排序；keyword 同时匹配名称、品牌、简介、标签、搜索词；分类与价格可叠加；pageSize 上限 60；categories 附带每类数量；recommendations 按销量、related 按同分类。
server/services/cart-购物车服务.js	getOrCreate 生成 cart_ + 12 位十六进制；decorate 把商品快照拼进购物车并算小计；addItem 累加前校验库存（40901 售罄 / 40902 不足）；updateItem 数量为 0 等价删除；remove / clear；单件上限 99。
server/services/orders-订单服务.js	下单时重新读购物车、校验商品与库存、用数据库单价重算金额；满 99 免运，否则 12 元；订单号 XM + 日期 + 6 位大写十六进制；扣库存、加销量、清空购物车、落盘；状态 pending/paid/cancelled，重复支付或取消已支付都会抛 409。
server/routes/products-商品接口.js	5 个 GET：列表、详情、相关推荐、分类、首页推荐。只把参数转交给 service。
server/routes/cart-购物车接口.js	GET 读车、POST 加购、PATCH 改数量、DELETE 删单项、DELETE 清空；cartId 从 body 或 query 取。
server/routes/orders-订单接口.js	POST 创建（返回 201）、GET 查询、POST 支付、POST 取消。

后端职责可以压成一句话

提供数据、维护状态、做校验和计算。前端传来的价格一律不信，订单金额一定用服务端数据重算；库存够不够、订单能不能支付，也都由服务端说了算。这句话是答辩时“后端做了什么”的标准答案。

五、前端 HTML：只搭骨架

四个页面都遵循同一个原则：HTML 里放结构和空的容器，具体内容由 JS 请求接口后填进去。所以查看源代码看不到商品，但打开页面能看到 26 件——这正是“数据不能硬编码在前端”的体现。

文件	页面结构与关键点
public/index-首页.html	顶部导航（Logo、三个图标入口、搜索框、购物车角标）；Hero 区（标题、两个按钮、三张拼贴图、三个统计数字）；分类区 #categoryGrid；推荐区 #recommendRow；列表区（筛选条、结果数量、商品网格 #productGrid、空状态、分页器）；侧拉购物车 #cartDrawer；结算弹窗 #checkoutModal；Toast 容器 #toastStack。
public/product-商品详情页.html	面包屑；加载失败提示块；左图右文布局：封面、名称、标签、评分与销量、价格盒（现价/原价/折扣）、数量步进器、加入购物车与立即购买、库存提示、规格表；商品介绍与卖点；相关推荐 #relatedGrid。
public/orders-我的订单页.html	带 ?id= 时显示单笔订单结果（成功/失败 Hero + 明细卡）；不带 id 时显示本地保存过的历史订单；顶部有订单号搜索框。
public/404-页面不存在.html	静态错误页，服务端找不到路径时返回它；提供回首页和看订单两个出口。

页面之间怎么跳

· 首页卡片 → product-商品详情页.html?id=p02；详情页返回面包屑或搜索框回首页。

· 加购不跳页，右侧抽屉滑出；点“去结算”弹窗；提交成功后跳 orders-我的订单页.html?id=订单号。

· 订单号同时写进 localStorage，所以“我的订单”能列出历史单；结果页刷新也不会丢。

六、前端 CSS：五层，后一层可以覆盖前一层

样式没有堆在一个文件里，而是按“基础 → 布局 → 组件 → 页面 → 补充”分层。好处是改按钮去 components，改首页大屏去 pages，不会在一千行里翻找。

文件	内容
css/base-设计变量与基础样式.css	全部设计变量（暖白底、陶土红、墨色文字、灰线、成功/警告/危险色、字号 clamp、间距刻度、圆角、阴影）；reset；衬线标题；按钮、输入框、价格等基础类。
css/layout-导航与页面布局.css	容器宽度；吸顶导航；图标 hover 上浮变色；Hero 两栏与拼贴；区块标题；商品网格 auto-fill minmax；分类网格；页脚；960/860/640/400 断点。
css/components-通用组件样式.css	商品卡片、库存角标、标签、筛选 chip、分类卡、侧拉抽屉与遮罩、购物车行与步进器、Toast、弹窗、分页、空状态、骨架屏、面包屑、评分、状态徽标。
css/pages-各页面专属样式.css	首页 Hero 标题强调与推荐卡；详情页两栏、价格盒、规格表、卖点列表；订单页结果 Hero、订单卡、金额行、订单项四列网格；404 大字。

文件	内容
css/extra-结算与补充样式.css	结算金额框、加购成功的小动画、订单操作按钮、窄屏补充规则。

响应式的三条做法

· **能自动就不写断点**：商品网格用 repeat(auto-fill, minmax(min(100%, 240px), 1fr))，宽度不够自动减列。

· **字号和间距用相对单位**：clamp() 让标题在手机和桌面之间平滑变化，避免某一档突然挤爆。

· **断点只处理结构变化**：960 详情改单列、860 导航只留图标、640 订单项换行、400 再收紧间距。在 390px 手机和 250% 缩放下都验证过没有横向滚动。

七、前端 JavaScript：一个文件一件事

9 个 ES 模块，按“工具 → 数据 → 组件 → 页面”组织。页面文件只负责编排，请求、状态、DOM 工具都可以被多个页面复用。

文件	作用与关键点
js/api-请求封装.js	所有 fetch 集中在这里：拼查询参数、12 秒 AbortController 超时、解析 JSON、code 不为 0 抛 ApiError、网络错误转成中文；按资源分成 products / cart / orders 三组；formatMoney 统一金额格式。
js/store-购物车状态.js	cartId 存在 localStorage 的 xm_cart_id；state 保存购物车快照；subscribe 立即回调一次并持续通知；busy 列表防止同一商品重复点击；add / updateQuantity / remove / clear 都是“先请求、成功后再更新状态”。
js/ui-通用界面工具.js	$ / $$ 选择器、escapeHtml 防注入、debounce、内联 SVG 图标、toast、抽屉与弹窗开关、Esc 和遮罩关闭、搜索框绑定、购物车角标绑定。
js/card-商品卡片.js	卡片 HTML 模板，首页、推荐位、详情页相关推荐共用；包含封面、库存角标、分类与品牌、标题、简介、标签、价格与原价、加购按钮；售罄时禁用按钮。
js/home-首页逻辑.js	把筛选条件存进 state 并同步到地址栏；加载分类、推荐、列表；用 requestSeq 防止旧请求覆盖新结果；骨架屏、空状态、错误提示、分页窗口；分类/价格/排序/搜索的事件绑定；加购成功的小动画。
js/product-详情页逻辑.js	从地址栏读 id；并行加载详情与相关推荐；渲染价格、折扣、库存文案、规格、介绍段落、卖点；数量步进器；加入购物车（留在本页）与立即购买（直接开结算）；相关推荐里也能直接加购。
js/cartdrawer-购物车抽屉.js	订阅 cartStore 渲染抽屉；事件委托处理加、减、删除、手动输入；空状态与底部合计；清空；去结算时关闭抽屉并打开结算弹窗。
js/checkout-结算下单.js	校验收货人、手机号、地址；计算并展示商品金额、运费、应付金额；POST 创建订单 → 延迟 900ms 模拟支付 → 记录订单号 → 清空本地购物车 → 跳结果页；提交中禁用按钮防止重复下单。
js/orders-订单页逻辑.js	有 id 时查单笔订单并渲染成功/失败 Hero 与明细；没有 id 时用 localStorage 里的订单号并发查询历史订单；支付、取消、复制订单号；订单号搜索。

模块之间的依赖关系

api 不依赖任何业务文件；store 依赖 api；ui 不依赖业务；card 依赖 api 与 ui；cartdrawer 依赖 store、ui、checkout；checkout 依赖 store、api、ui；home / product / orders 是三个页面入口，负责把上述模块组装起来。依赖是单向的，所以不会出现循环引用。

八、数据、脚本与文档

文件	作用
server/data/products-商品数据.json	26 件商品：id、名称、品牌、分类、子分类、价格、原价、库存、评分、销量、emoji、上架时间、标签、搜索词、简介、详细介绍、卖点数组、封面路径。
server/data/categories-商品分类.json	6 个分类：key、名称、emoji、一句描述；分类名与商品 category 字段对应。
server/data/carts-购物车数据.json	运行时生成；每项含 id、items、createdAt、updatedAt。
server/data/orders-订单数据.json	运行时生成；保存订单号、商品快照、金额、收货信息、状态与时间。
scripts/gen-covers-生成商品封面.js	用商品 id 做种子生成确定性的柔和渐变与色块，按分类配色，中间放 emoji；输出 800×600 SVG 到 public/assets/products，并把 cover 字段写回商品 JSON。
docs/接口文档-前后端约定.md	14 个接口的参数、响应示例、错误码表、排查步骤。
docs/演示脚本-答辩怎么讲.md	8 分钟演示顺序、三条链路的讲法、高频追问与回答。
docs/screenshots/	**首页、列表、详情、购物车、结算、订单、手机端截图。**
package.json	name、start / dev / covers 三个脚本、Node 版本要求。
README.md	项目介绍、快速开始、功能与考核点对照、目录结构、接口一览。
.gitignore	忽略 node_modules、日志、本地验收脚本、运行时生成的购物车与订单。

为什么不把商品写在前端

如果 26 件商品直接写死在 HTML 或 JS 里，搜索、筛选、分页就只能在浏览器里做，价格和库存也随便改。现在商品在服务端，前端只拿到当前页需要的字段；接口还能被其他客户端复用，这也是后面接 AI Agent 工具的基础。

九、三条关键链路（答辩重点）

面试官通常不会让你逐行讲代码，而是问“这个功能怎么实现的”。下面三条链路覆盖了大部分问题。

链路一：搜索与筛选

· 用户输入关键词或点分类 chip，home-首页逻辑.js 更新 state，并用 history.replaceState 同步到地址栏。

· api.products.list() 把 state 拼成查询字符串，请求 /api/products。

· catalog 服务依次做：关键词过滤 → 分类过滤 → 价格过滤 → 排序 → 分页，返回 list 与 total。

· 页面先显示骨架屏，拿到结果后渲染卡片、结果数量、分页器；没有结果时显示空状态和“清空筛选”。

· 快速连续点击时，requestSeq 只让最后一次请求的结果落地，避免旧数据覆盖新数据。

链路二：加入购物车

· 列表和详情页都用事件委托监听 [data-add]，拿到商品 id。

· cartStore.add() 发 POST /api/cart/items，只带 cartId、productId、quantity。

· 服务端找到或新建购物车，校验库存，累加数量，返回整辆车的最新快照。

· cartStore 用快照更新 state 并通知订阅者；顶栏角标、抽屉、结算弹窗同时重绘。

· 购物车放在服务端，是为了下单时还能再校验一次库存和价格，前端改不了金额。

链路三：下单与模拟支付

· 结算弹窗先做格式校验：收货人至少 2 字、手机号 1 开头 11 位、地址至少 5 字。

· POST /api/orders 只传 cartId、收货信息和备注，不传金额。

· 服务端重新读购物车、校验库存、按数据库单价重算商品金额与运费、生成订单号、扣库存、清空购物车。

· 前端拿到 201 后调用支付接口，延迟 900ms 模拟支付过程，把订单号写进 localStorage。

· 跳转结果页，用订单号 GET 订单详情，渲染状态、明细、金额、收货信息和下单时间。

一个容易被追问的点

“前端校验了，后端为什么还要校验？”——前端校验是为了体验，后端校验是为了正确性。任何人都能绕过前端直接调接口，所以库存、价格、订单状态必须在服务端再判断一次。项目里的 40901 / 40902 / 40903 就是这些判断的结果。

十、这个项目用到了哪些知识点

下表可以当成“我学到了什么”的对照表。左列是知识点，右列是它在项目里的具体落点。

知识点	在项目里的落点
HTML 语义化与表单	三个页面的 header/main/section/footer；结算表单的 label、input、required
CSS 变量与 clamp	base 文件集中管理颜色、字号、间距；标题字号随视口平滑变化
Flex 与 Grid	导航、卡片内部用 flex；商品网格、订单项用 grid
媒体查询	960 / 860 / 640 / 400 四个断点处理结构变化
DOM 查询与修改	首页渲染商品卡片、详情页填字段、订单页拼明细
classList	抽屉与弹窗的 is-open、按钮的 is-added、Toast 的进出场
事件监听与事件委托	搜索、筛选、排序；列表加购；抽屉加减删除；订单页支付/取消/复制
表单取值与校验	结算弹窗读取收货人、手机号、地址并逐项提示
localStorage	保存 cartId 与订单号，换页面仍能找回
数组方法	map 生成标签、join 拼 HTML、find 查商品、reduce 算合计
闭包与防抖	debounce 包住搜索；requestSeq 保存在模块作用域里
Promise 与 async/await	api 封装、首页并行加载分类与推荐、订单页并发查历史单
try/catch 与错误提示	请求失败转成中文 Toast；订单不存在时显示错误块
ES Module	9 个 JS 文件用 import/export 组织，浏览器原生支持
fetch / HTTP	GET 查、POST 建、PATCH 改、DELETE 删；查询参数与 JSON 请求体
Node 与 JSON 文件	http 建服务、fs 读写 JSON、path 拼路径、crypto 生成随机 id
Git	分支、提交、远程仓库（提交到 GitHub 后补链接）

十一、演示顺序与排查清单

演示时不要一上来就讲代码，先让页面把流程走完，再回头解释。

八分钟演示顺序

· 首页：导航图标 hover、搜索框、购物车角标、分类入口、推荐位、列表与分页。
· 搜索“耳机”，再叠加分类和价格，指出地址栏同步、结果数量变化。
· 进详情，指出价格/原价/折扣、库存、规格、卖点、相关推荐；数量改成 2 加购。
· 抽屉里改数量、删除、清空，指出单项小计与总金额。
· 去结算，先填错手机号展示校验，再填对提交，指出 900ms 支付过程。
· 结果页指出订单号、状态、明细、金额、收货信息；刷新证明数据还在。
· 我的订单看历史单，演示未支付订单取消。
· 拖窄到 390px 或 250% 缩放，指出导航只剩图标、网格自动减列、没有横向滚动。

出问题时的排查顺序

· Network 面板看 Status：4xx 是参数或状态问题，5xx 是后端代码问题。
· 看 Response 的 code 与 message，对照接口文档的错误码表。
· 看 Payload / Query String，确认参数名、类型、大小写。
· 404 且返回 HTML，说明路径没进接口，检查 /api 前缀与拼写。
· Failed to fetch，检查服务是否启动、端口是否一致、是否直接双击了 HTML 文件。
· 服务端终端有每条请求的方法、路径、状态码、耗时，可与前端时间对得上。

主动说的三个取舍

· 不用框架：页面少、数据量小，原生更透明；代价是状态和路由要自己管。
· JSON 当数据库：零依赖、易读；代价是并发写入和大数据量要换数据库。
· 购物车放服务端：下单能复算金额；代价是每次操作多一次请求。

十二、接下来可以做什么

当前版本已经覆盖考核的 3.1–3.9、第四部分接口要求和第五部分数据要求。下面这些是自然的下一步，不会推翻现有结构。

方向	要做的事	会改到哪些文件
注册 / 登录	用户表、密码哈希、会话或 JWT；订单按用户隔离；我的订单改为服务端列表接口	新增 users 数据与 auth 服务；orders 增加 userId；前端加登录页与请求头
订单列表接口	GET /api/orders?phone=... 或按用户查，替代 localStorage 逐个查询	orders-订单服务.js、orders-订单接口.js、orders-订单页逻辑.js
换数据库	SQLite / MySQL 替换 JSON 文件，把筛选与分页下推到 SQL，加索引	store-JSON数据存储.js 换成 store-SQLite数据存储.js，services 基本不动
AI Agent 导购	对话、searchProducts 工具、多轮约束、写操作确认、流式输出；复用 services	新增 agent 目录与接口；前端加对话面板；本版未实现
部署上线	静态资源与接口同域部署；或用 Nginx 反代；加 HTTPS 与进程守护	package.json、README、部署脚本

最后一句

这份说明书里的每个文件都不是为了“显得工程化”而拆的：它们分别解决了“请求怎么发”、“状态放在哪”、“规则写在哪”、“数据从哪来”四个问题。当你能不看代码说清这四个问题，这个项目就真正属于你了。

—— 小满商城 · 文件说明书 · 完 ——